

MANUAL26. УРОК 10. БАГ-РЕПОРТ: КАК ОПИСЫВАТЬ ДЕФЕКТ ТАК, ЧТОБЫ ЕГО БЫСТРО ИСПРАВИЛИ.

На прошлом уроке мы учились находить нестабильные сценарии через exploratory-подход: чартер, таймбокс, заметки. Но найденный дефект сам по себе еще не решает проблему команды. Если его описать размыто, разработка потратит время на уточнения, не сможет воспроизвести проблему или исправит не то. Поэтому в этом уроке фокус не на поиске, а на качественной передаче информации.

Что нового в этой теме

Тема урока — баг-репорт. Мы разбираем, как оформить дефект в понятном и воспроизводимом виде: какие поля обязательны, как писать шаги, что считать

хорошим ожидаемым и фактическим результатом, какие ошибки в формулировках чаще всего ломают коммуникацию.

Зачем качественный баг-репорт и почему он критичен для команды?

Найти баг — это только полдела. Вторая половина куда важнее: передать дефект так, чтобы команда смогла быстро его воспроизвести и починить. Слабый баг-репорт легко «прячет» даже серьезную проблему.

Когда в репорте нет контекста, шаги расплывчаты, а факт описан общими словами, разработка тратит время на уточнения. Появляются лишние комментарии, созвоны и риск статуса «не воспроизводится», хотя проблема реально есть.

Хороший баг-репорт — это не красивый текст, а рабочий инструмент. Его можно открыть без автора, пройти шаги и увидеть тот же результат. Чем выше проверяемость и конкретика, тем быстрее команда принимает решение и

тем меньше потерь перед релизом.

Что такое хороший баг-репорт простыми словами?

Хороший баг-репорт — это отчет, из которого ясно:

1. где и в каком контексте произошла проблема;
2. что именно нужно сделать, чтобы ее воспроизвести;
3. что ожидалось по требованиям;
4. что произошло фактически;
5. какой риск это несет пользователю.

Если хотя бы один из этих пунктов размыт, вероятность «шумной коммуникации» резко растет.

Базовая структура баг-репорта

Краткий заголовок

Должен передавать суть дефекта без лишних эмоций. Формат «где + что ломается + при каком действии» обычно работает хорошо.

Окружение

Сборка, платформа, версия, устройство, режим — все, что влияет на воспроизведение.

Предусловия

Состояние системы до начала шагов: аккаунт, настройки, статус, данные.

Шаги воспроизведения

Четкая последовательность действий. Без «сделайте обычные действия». Любой участник команды должен пройти их без догадок.

Ожидаемый результат

Что система должна сделать по требованиям.

Фактический результат

Что система сделала на самом деле.

Дополнительные материалы

Скриншот, видео, лог, trace, код ошибки — все, что ускоряет анализ.

Риск/влияние

Короткое описание последствий для пользователя и релиза.

Fortnite

Представим, что в Fortnite после завершения события игрок должен получить награду и увидеть обновленный баланс. Во время теста вы замечаете: награда начисляется, но баланс в интерфейсе не обновляется до перезахода.

Плохой отчет выглядел бы так: «Награды багуются, срочно чинить». Из него непонятно почти ничего.

Хороший отчет:

- **заголовок:** «Fortnite: баланс награды не обновляется на экране события до перезахода после завершения матча»;
- **окружение:** build X, платформа Y, режим Z;
- **предусловие:** игрок участвует в событии, имеет доступ к награде;
- **шаги:** войти в событие -> завершить матч -> открыть экран наград;
- **ожидаемо:** баланс обновлен сразу;

- **фактически:** баланс старый до ручного перезахода;
- **риск:** игрок думает, что награда потеряна, падает доверие.

Такой баг-репорт уже можно быстро взять в работу.

Как писать шаги воспроизведения так, чтобы не было «не воспроизводится»?

1. Первое правило — пишите шаги как маршрут, а не как пересказ. Каждый шаг должен быть отдельным и наблюдаемым.
2. Второе — не пропускайте важный контекст. Если дефект проявляется только после переключения режима, это обязательно должно быть в шагах.
3. Третье — не смешивайте шаги и выводы. В шагах пишем действия. Выводы выносим в блок «фактический результат» и «риск».
4. Четвертое — проверяйте свой репорт на себе: откройте его через 10 минут и попробуйте повторить сценарий, следуя только записанному тексту. Если самому сложно —

разработчику будет еще сложнее.

Ожидаемый и фактический результат: где чаще всего ломается смысл

Ожидаемый результат должен быть связан с требованиями или договоренностями команды. Формулировка «должно работать нормально» не помогает.

Фактический результат должен описывать наблюдаемое поведение, а не эмоцию. Вместо «все зависло» лучше «после шага 4 экран не обновляется 20 секунд, кнопки неактивны, приложение не падает».

Чем выше наблюдаемость и конкретика, тем меньше споров и быстрее исправление.

Частые ошибки в баг-репортах

1. Первая ошибка — эмоциональные формулировки вместо фактов.
2. Вторая — отсутствие условий воспроизведения.

3. Третья — пропуск шагов «между строк».
4. Четвертая — ожидание и факт написаны одинаково или расплывчато.
5. Пятая — нет оценки риска для пользователя.
6. Шестая — не приложены материалы там, где без них сложно понять проблему.

Все эти ошибки лечатся одним принципом: писать так, чтобы баг-репорт работал без автора.

Как баг-репорт связан с quality mindset

Сильный баг-репорт показывает не только «что сломано», но и «почему это важно». Это уровень мышления QA, который смотрит на продукт глазами пользователя и команды одновременно.

Когда вы добавляете короткую и точную оценку риска, команда быстрее понимает приоритет реакции. Когда вы фиксируете шаги воспроизведения качественно, команда быстрее исправляет. Когда вы прикладываете артефакты, команда быстрее локализует причину.

Поэтому качественный баг-репорт — это не формальность, а

инструмент ускорения релиза и снижения конфликтов внутри команды.

Итоги занятия

На этом уроке вы разобрали, как оформлять дефекты так, чтобы ими было удобно работать: заголовок, окружение, шаги, ожидание, факт, риск и артефакты. Теперь вы умеете не просто находить проблему, но и передавать ее в команду в рабочем виде.

Профессиональный вывод: ценность QA не в количестве «созданных багов», а в качестве информации, которая помогает команде быстро и точно принимать решения.

Глоссарий к занятию

1. **Баг-репорт** – это структурированное описание дефекта, достаточное для воспроизведения и анализа.
2. **Шаги воспроизведения** – это последовательность действий, при которой дефект проявляется повторяемо.
3. **Ожидаемый результат** – это поведение системы,

соответствующее требованиям
или договоренностям.

4. **Фактический результат** –

это реальное наблюдаемое
поведение системы при
выполнении шагов.

5. **Риск пользователя** – это

последствие дефекта для
цели, опыта и доверия
пользователя.